Week 4: Requirements

G1: Identify Stakeholders First

- ID all stakeholder groups before generating any requirements, then run separate prompts per group
- Practitioners in unfamiliar domains don't know to look for non-user stakeholders; models reproduce the same blind spot unless explicitly told
- Ask model to list groups including operators / maintainers / auditors, not just end-users; one-sentence concern each
- Don't merge personas into one prompt; treat list as starting point, add / remove roles
- Cross-check model's stakeholder list against domain standards or regulations

G2: Flipped Interaction Pattern

- Have model drive elicitation by asking targeted questions one at a time instead of front-loading your spec
- Thin-prompt requirements are abstract / invisible; structured questioning grounds output in confirmed intent.
- Open with "ask me questions one at a time... start with the most important."
- Don't volunteer long descriptions upfront; confirm understanding after each answer before next question.
- After session, ask model to list unconfirmed assumptions; verify every req. traces to a given answer.

G3: Cognitive Verifier Pattern

- For vague / overloaded req., have model list sub-questions, answer them, and let it write final req.
- Single-shot prompts fill gaps with statistically common answers; sub-questions surface ambiguity early.
- Ask the model to list sub-questions needed... instruct it to not answer it themselves
- Answer sub-questions in follow-up; explicitly reference your answers when requesting final requirement
- If model introduces an undefined term, make it flag assumption + propose two alternatives to choose from

G4: Provide a Structured Output Template

- Paste exact SRS template into prompt rather than asking for free prose
- Pre-defined templates significantly improve output quality; free-form generation needs heavy revision
- Paste org's actual template verbatim - don't just say "IEEE 830", no single standard layout exists
- Specify field values explicitly (Eg: Priority: High / Medium / Low); one req. per template block
- Flag blank / placeholder fields; confirm Verification method is executable & Priority has bus. rationale

G5: Detect Conflicts & Undefined Terms

- Once draft req. are complete, feed full list to model to find contradictions & undefined shared terms
- Validation is the most reliable LLM contribution; "find contradictions" is narrow, not generative
- Prompt: "Identify contradicting pairs; list shared terms lacking definitions; output as table rows"
- Use seperate output tables for contradictions & undefined terms; include FRs & NFRs together
- Hold stakeholder decision meeting per conflict; FR v/s NFR conflicts get top priority

G6: Supply a Worked Example for NFRs

- Include one concrete, measurable NFR example directly in the prompt when generating NFRs
- Without an example, the model defaults to vague adjectives; few-shot transfers measurability
- Format as "[Quality attribute]... [Measurable condition]... [Verification Method]"
- Delete any NFR lacking numeric threshold / named standard; confirm security / availability NFRs exist for auth or external-data FRs.

Week 05: Code Generation

G1: Size Each Subtask

- Keep output to 10-50 lines per prompt; reject larger outputs and ask for decomposition
- Defect detection drops sharply past 200-400 lines/hour; AI code is harder to review than code you haven't reviewed it
- State expected size prompt; decompose before requesting code, not after
- One concept per prompt | - Read every line aloud; name what each branch handles - if you can't,

G2: Ask for Plans as Options, Not Answers

- Request 2-3 plans with explicit trade offs; require model to state strengths / weaknesses of each
- Plan-then-implement: 25.4%. Pass@1; role-based decomposition: 29.9-47.1%.; planning errors compound
- Ask for 2-3 complete plans; reject cosmetic differences; pick one explicitly before proceeding
- For each step, ask model to name alternatives considered & concrete reason for choice into prompt
- Verify chosen plan covers all requirements, not just happy path; promote unverifiable assumptions

G3: Ground Prompts in Codebase

- Show function signatures, type definitions, naming conventions, existing libraries before requesting code
- Surrounding code is strongest lever for correctness; repo context improves completion >10%.
- Paste signatures of functions code will call / be called by; list existing libraries + preference
- Include 5-10 lines from similar function as style reference; name conventions (case, logging, types)
- Read imports first; check new dependencies already exist; compare proposals against current ones

G4: Drive Generation with Executable Tests

- Write 3-6 tests covering normal, boundary & edge cases before requesting code; require passing implementation
- Tests + problem beats problem alone; test-driven reduces cognitive load + improves evaluation
- Hand model signature and tests together; ask for implementation that passes and explains assumptions
- If model generates tests, review each one before running | - Run tests yourself; avoid overfitting

G5: Specify Pre/Post-Conditions & Edge Cases

- Before asking for code, write input types / ranges, output contract, exceptions, edge-case behaviour
- Specifications drive 4 of 5 top prompt improvements; I/O format + pre/post conditions rated usefullest
- State each param type / shape / range; explicitly say what empty, zero, negative, large inputs must do
- State output type / shape / invariants; name exceptions + fire triggers ("Raise ValueError if x < 0")
- For every stated edge case, run a test; confirm exception types; silent acceptance = spec violation

G6: Model Self-Critique Before Execution

- After code generation, ask model to critique against spec before running don't trust
- Feedback + refinement beats single pass; quality drops on harder problems - evaluate critique,
- Forbid rewriting in critique step; for each item ask "if wrong what breaks" to test real consequence
- Ask follow-up per item to gauge importance; this signals whether item actually matters
- Read each item as hypothesis, not fact; decide real / model-wrong / non-issue; if critique misses visible problem, be skeptical of rest; then specify what to fix

YASH H. BARVE

yhbarve20930353

WEEK-06: CODE SUMMARIZATION

G1: Specify Summary Intent

- Name the intent explicitly before asking [Eg: what, why, how-is-it done]
- LLMs produce high-quality summaries across all five categories only when intent is explicit; unconditioned prompts default to vague "what it does" and miss what readers actually need
- State intent clearly and tell the model who the reader is. — prompts tend to be long
- Ask for one intent at a time unless you genuinely want a multi-part summary; blended
- Verify output matches the intent you requested; reject summaries that restate syntax line-line.

G2: Surface Semantics Facts the AI Cannot See

- Attach semantic facts to prompt — repo name, function name, identifiers, pre/post conditions. — intent-carrying names and flows
- Adding semantic facts improved summary quality across 3 LLMs; explicit labeling fowusses attention on
- Include repo/module name & fully qualified function name so model grounds vocabulary in the project.
- List non-trivial identifiers with brief notes if meaning isn't obvious; state pre/post conditions in 1-2 lines
- Verify project-specific terms from the facts in the summary; flag invented behaviour & generic synonyms

G3: Verify Against Behaviour, Not Plausibility

- Treat AI-generated summary as a testable claim about code behaviour; verify via execution / unit tests
- Manual inspection of 540 LLM-generated comments found inaccuracies in 1/5 cases even for best models
- Ask for claims phrased as concrete behaviors rather than vague prose; request claims as bullet list at the end
- For edge cases, ask the model to state expected behaviour explicitly. — with extra suspicion
- Confirm the most load-bearing claim by running the function on matching input, treat unverified claims

G4: Don't Trust Surface Fluency

- Judge summary by what it lets readers do, not how fluent or professional it sounds — generic style
- Participants with human-written summaries performed significantly better on comprehension.
- Tell the model the task that the summary must support, so the output is shaped by the task & not
- Ask for concrete, checkable facts instead of adjectives; prefer short dense summaries over long ones
- Run a quick comprehension test. Ignore model's confidence language when deciding whether to keep.

G5: Adapt Prompt to Language to Prompt

- Change how you prompt based on the programming language & paradigm of the code.
- Evaluated LLMs performed worse on logic-programmic languages; perf. varies among paradigms
- Name language & paradigm explicitly so model doesn't default to imperative vocabulary
- For logic/functional/declarative code, include the meaning you want captured rather than asking for generic description; for less common languages give a one-line behavioral anchor to stabilize output.
- Check that summary uses paradigm-appropriate vocab.; be extra skeptical with less common languages.

G6: Seed Project Style with Local Examples

- Include 2-3 example summaries from the codebase when asking for a summary inside an existing project
- Few-shot prompting significantly improved quality over zero-shot & cross-project few-shot prompting
- Pick 2-3 existing summaries from same/related module & paste as (code, summary) pair in the prompt
- Put function to summarize last so the model uses examples provided as a style template
- Compare output's length / tone / terminologies against examples; large deviations → examples were ignored

G1: Narrow the Bug Before You Prompt

- Shrink problem along three axes: history, input, code
- Bounding failure unblocks debugging fastest; models guess symptoms without it — on wrong value
- Use git bisect if regression; minimize input by halving; minimize code to only backward slice
- Deleted unrelated imports & helpers. Paste narrowed artifact, not whole project — in prose
- If AI asks clarifying Qs, it signals reproducer too big / value — shrink further, don't explain

G2: Prompt with Observed Runtime State

- State expected invariant; set conditional breakpoint on negation, run, print variables, paste — + state + source
- Concrete state = main defense v/s hallucinated APIs; LLMs diagnose execution better than simulation
- Write invariant first, before debugger
- Copy debugger output verbatim, for large structures include summary + suspicious elements only
- AI explanation must reference specific observed values you pasted; push back otherwise

G3: Get Ranked Hypotheses, Then You Pick

- Request 3+ ranked hypotheses with explicit reasons tied to concrete evidence; you pick which one to test
- Handing pick to model recreates automation-bias; your reason surfaces private context AI can't know
- Each hypothesis needs prediction of what would hold if true; forbid AI from proposing a patch
- Write concrete pick reason; ask AI to push back; sycophantic agreement = re-prompt signal
- Ask for cheapest experiment distinguishing your hypothesis from closest alternative; not one that confirms that pick.

G4: Land Every Fix with Test + Invariant

- Before accepting patch, test currently failing on buggy code, & executable invariant assertion in function
- APR most often suppress symptom not remove cause — hard to detect from test results
- Ask for failing test, invariant, minimal patch; ask for 2 adversarial tests naive fix wouldn't pass
- Supply language + library versions to reduce hallucinated API calls ; watch for overfitting patterns
- Run failing test on unpatched code, confirm right failure reason; apply patch; re-run suite + adv. tests

G5: Classify Root Cause + Symptom Fix

- Before reading fix, ask "Remove cause from trace, or add guard preventing symptom observation?"
- LLM performance drops sharply on semantic bugs; confident AI patch worsens this
- Ask AI to classify in one sentence: root-cause removal v/s symptom guard?
- If guard, ask it to name root cause it protects & why not fix directly; if AI can't classify, re-examine
- Recognize 4 patterns: Broad try/accept, None guard, special-case, logging-continue; trace backward

G6: Keep a Debugging Log

- Write every hypothesis tested, experiment run, fix tried + result + one-line reason; feed log in prompts
- Losing track causes sessions to spiral; model re-proposes fixes; log prevents repetition
- Start prompt w/ short log; Ask "Given what's ruled out, next hypothesis + cheapest experiment?"
- If chat > handful turns, start fresh with only log + minimal reproducer — on handoff
- Point out AI re-suggestions explicitly; update log immediately after each attempt; share log

Week 09: TESTING

G1: Anchor Assertions To Spec. (Not Code)

- Supply spec. as basis for expected values — don't let model derive them solely from reading the implementation
- A model reading only code repeats its bugs as expected — oracle deficiency; top practitioner concern
- Give docstring / spec / worked example; instruct model to "derive from spec, not by tracing code"
- Numeric → Give formula; strings → give reference doc example; not implementation
- Ask "could this be read off the implementation" → verify v/s spec; test against known-buggy version (pass = red flag)

G2: Boundary / Equivalence - Class Coverage

- List boundary conditions / equivalence classes explicitly in the prompt; don't assume model applies structured design on its own
- Reasoning: LLMs default to happy-path; branch coverage remains weakest dimension without guidance
- Prompting: List classes / boundaries yourself first, paste verbatim into prompt
- Prompting: Numeric → name exact values (0, 1, 99, 100, 101); strings → empty; mid-len, max-len, invalid char
- Review: Boundaries tested at the edge itself; invalid-class tests assert rejection; not just "no crash"

G3: Specify Mocking / Stubbing

- Explicitly name each dependency and specify mock / stub — don't let model decide isolation on its own
- Reasoning: LLMs unreliable alone — skip mocking, use unrealistic mocks, or mock wrong boundary
- Prompting: name dependency + framework + injection point; describe stub behavior per variant
- Prompting: Ask model to assert on mock interactions (call count / args)
- Review: No real network / DB / file calls; mock sits at close edge with realistic return values

G4: Properties / Metamorphic Relations

- For hard-to-derive outputs, prompt for property-based tests / metamorphic relations, not concrete assertions
- Reasoning: sidesteps oracle problem — express what's always true rather than exact output
- Prompting: Explicitly request property-based tests + name framework; have model enumerate invariants
- Prompting: Use categories — round-trip, idempotency, monotonicity, metamorphic ("f(x) v/s f(y)")
- Review: run 500-1000+ samples, ensure edge cases generated, check properties aren't vacuous

G5: Regression Tests from Bug Reports / Diffs

- Prompt with bug description or diff, ask for test that would've caught it — write before / alongside fix
- Reasoning: must fail pre-fix, pass post-fix — general coverage-driven generation won't fix this
- Feed bug report / diff verbatim (repro steps + observed v/s expected); instruct "must FAIL before fix"
- Ask for comment explaining why test catches the bug — confirms correct understanding
- Verify fails pre-fix checkout, passes post-fix, narrow to the specific input

G6: Mutation Testing > Coverage Alone

- Don't use line coverage as sole quality signal — use mutation testing to confirm suite detects fault
- Reasoning: AI assertions often syntactically fine but semantically weak — coverage ≠ fault detection
- Prompting: Target specific mutations you list (eg: > → >=); annotate assertions with fault they catch
- Prompting: Numeric → request tests at off-by-one sensitive values
- Review: Run mutmut / PIT, feed surviving mutants back for targeted tests; score $\frac{\text{mutants killed}}{\text{total mutants}}$ ≤ 70% indicates "write more"; ignore 100% coverage claims alone [mutmut: Python; PITest: Java]

WEEK-10: Code Review

G2: Prepare Small, Well-described Changes

- Keep each PR to one logical unit; write clear descriptions before AI review
- Small reviews (200 LOC/hr) catch ~2/3 design defects; LLM precision < 10% on large diffs.
- Prompting: One logical change per PR — split refactor / rename / feature into separate PRs
- Prompting: If diff unreadable in 30 min, split into stacked PRs; ask AI to suggest commit order
- Include what / why / how tested / what to check; name invariants to preserve; state rejected alternatives

G2: Feed the Reviewer the Surrounding Code

- Send enclosing function / class + imports + call site; not just changed lines
- Reasoning: Insufficient context → false positives; AST-boundary context yields more reliable comments
- Prompting: Paste full enclosing function / class; mark challenged lines (+/-)
- Prompting: Include imports / type defs; for public functions, add at least one call site
- Flag AI comments referencing code not provided (guesses); re-run after adding missing context

G3: Run the Review as Two Passes

- Seperate comprehension pass (explain the change) from rubric pass (evaluate against criteria)
- Defect detection is downstream of comprehension — wrong summary poisons all critique
- Pass 1: Ask AI to describe change in 3-5 sentences (no bugs/issues); infer intent + invariants
- Pass 2: Short imperative rules under headings (security / perf / error handling / style); scoped & specific
- Read summary first; fix prompt if wrong before rubric pass; include negative rules; store rubric in version control

G4: Treat Findings As Hypothesis

- Treat every AI comment as an unverified claim, not a verdict
- LLM precision < 10% on full-PR reviews; priming with prior comments causes reviewers to over-anchor
- Prompting: Require each finding to cite exact lines + evidence; ask for minimal failing example
- Review: Locate evidence yourself. No evidence = suspect; classify as confirmed / style / false-positive
- Never batch-accept "apply all"; for second opinion, run blind first, then a primed consolidation pass - only

G5: Own the Merge, Own the Learning

- Human keeps merge decision, approval & accountability; AI can't own these
- Reasoning: Cannot be accountable to an LLM; quality drivers are interpersonal; automation bias causes mixed errors
- Ask for evidence-based observations, not verdicts; ask AI to rank confidence & flag guesses
- Ask for alternative approaches, not just defects; use AI's "why it matters" as raw material
- Read the diff yourself first; write comments that explain why; respond to author directly

G6: Heightened Scrutiny for AI-authored / Security-sensitive code

- Default AI review calibrated for routine code — security sensitive / AI authored changes need more
- AI authored PRs ~1.7x more issues; malicious-PR detectors hit 99.3% with static analysis + named threat list
- PR mentions AI co-authorship, touches auth/crypto/IO/CI, first-time contributor or unusually large/broad
- Give AI a named threat list rather than a vague "is this safe?"
- Run static analysis / dependency scanners first; ask AI to verify every external symbol exists;
  human always reads auth / crypto / subprocess / CI changes regardless of tool output

WEEK-11 : PERFORMANCE

G1: Feed Profiler Output Back Into Prompt

- Run a profiler first; give the model the report; not just "optimize this"
- Profile-guided self-optimization loops cut runtime ~87%, memory ~91% w/ negligible correctness loss
- Prompting: Paste top 10-20 lines of cProfile / perf / gprof / Valgrind output
- State bottleneck explicitly - function name, % runtime, call count, input size / access pattern
- Review: Re-run profiler after; reject rewrite unless bottleneck's share visibly dropped

G2: Verify Algorithmic Complexity of Generated Code

- Never assume correct output is also efficient — demand complexity analysis
- Reasoning: GPT-4 pass@1 0.831 v/s eff@1 0.454 — half of correct sols. aren't runtime-competitive
- Prompting: Specify target complexity upfront (Eg: "O(nlogn) time, O(n) space")
- Ask model to identify best-known complexity before coding, then justify after
- Check hidden O(n) ops; empirically test n = 1000, 2000, 4000 scaling

G3: Provide Input Characteristics to Prompt Algorithm Switching

- Give typical input distribution so model can build hybrid fast / slow-path logic
- Models default to one general algorithm; won't hybridize without explicit distribution info.
- State input size frequency (eg: "90% n < 32; 10% n up to 100K") — crossover point
- Prompting: Request named branches (_small_case / _general_case) with per-branch complexity +
- Test boundary cases (threshold - 1, threshold, threshold + 1) & measure actual empirical crossover

G4: Provide a Fast Reference Implementation

- Give a known-fast example / pattern; frame task as slow → fast implementation
- PIE-prompted models hit 6.86x mean speed up v/s laziest-default transforms otherwise
- Supply SLOW + FAST reference pair explicitly labeled, using desired strategy (DP/hash)
- Ask model to narrate mapping between reference & new implementation — can't be a constant
- Confirm transformation actually applied (not verbatim copy); speedup should scale with n

G5: Specify Memory Hierarchy Constraints for Low-Level Optimization

- State cache size, data type, access pattern before requesting C++ loop optimization
- Without this, models produce cache-hostile, readability-optimized code
- Name cache-size, line-width, and data footprint
- For SIMD, restrict to standard intrinsics / OpenMP programs and require justification per — intrinsic
- Verify with 'perf stat -e cache-misses', and check compiled binary actually vectorizes

G6: Audit Parallelism Proposals for Correctness Before Measuring Speedup

- Treat AI-added parallelism as a correctness-critical draft, not a finished optimization
- Parallel pass@1 only 37.8 v/s 84.1 serial - races / missing syncs are common silent failures
- Name shared mutable state explicitly & required sync strategy
- For Python, clarify GIL (Global Interpretor Lock) - ProcessPoolExecutor for CPU-Bound, ThreadPoolExecutor for I/O-Bound
- Run parallel v/s serial on deterministic input ~10x to catch non-deterministic races; check hardcoded thread counts

WEEK-12: SECURITY

G1: Enumerate Candidate Threat Models Before Writing Code

- Have AI list applicable threats w/ CWE IDs + reasoning before generating security-sensitive code
- LLMs miss context-dependent flaws (eg: broken access control) invisible from code alone
- Describe system plainly; request Threat / CWE ID / Reason table
- Use STRIDE categories; ask for plausible threats, not a generic checklist
- Treat list as candidate - verify CWE IDs, add omitted threats, carry confirmed list forward

G2: Declare the Threat Model in Every Security-Sensitive Prompt

- Name specific vulnerabilities to defend against + exact libraries / standards in the prompt itself
- Omitting constraints yields functionally correct code that silently skips defenses
- Paste confirmed threat list as numbered requirements; name each vulnerability by CWE ID
- Specify exact mechanism; include full contract in one prompt, not follow-up
- Confirm every named defense appears in output, check error messages don't leak internal info.

G3: Generate Exploit-Style Tests From the Threat Model

- After hardening the code, have the AI write one test per CWE from the threat model
- Unanchored "make it more secure" iteration raises critical vulnerabilities by 37.6%.
- Paste threat model table alongside code; one test per CWE, labeled by ID
- Each test = exploit attempt + assertion; use Hypothesis for free-form string inputs
- Confirm test fails on naive / passes on hardened; assertions go beyond status codes

G4: Prompt For Hardened Infrastructure Configs & Verify Before Development

- Explicitly name security constraints when generating Docker / DB / cloud configs; verify before deploying
- AI generated IAC defaults to open ports / wildcard IAM / public resources; passes app-level SAST checks
- State deployment environment + constraints; Ask model to justify every exposed port / permission
- Check bind addresses; auth / default creds replaced, IAM wildcards / open CIDRs

G5: Verify Every AI-Suggested Dependency Before Installing

- Manually confirm any AI-suggested package exists on official registering before installing.
- "Slopsquatting": Attackers register packages matching names AI commonly hallucinates — confirmation
- Require well-known packages (1M+ monthly downloads); ask for pinned versions & existence
- If unsure, ask model directly "does this package exist on PyPI/npm?"
- Search registry, run pip/npm audit post-install; enable Dependabot/Snyk; pin CVE-free versions

G6: Anchor LLM Vulnerability Review with SAST Tool Output

- Run static analysis first, paste its output into the prompt instead of asking AI to "review for security issues"
- CodeQL-grounded prompts cut residual vulnerability rate
- Run Bandit / Semgrep / CodeQL first; paste full output verbatim in the prompt
- Ask model to explain why flagged line is vulnerable before requesting fix; max 2 iteration rounds — w/o human checkpoint
- Re-run same SAST Tool on the fix; check no new vulnerability is introduced; confirm fix is logically equivalent to original

SAST: Static Application Security Testing